

Apache Kafka

Aurreratua

Kafka klusterra, AEMET Bronze/Silver/Gold pipeline-a, NiFi integrazioa eta Kafka Connect.

Zikloa Adimen Artifiziala eta Big Data

Modulua Big Data Aplikatua



01

2. Kasua — Kafka Klusterra

Kafka Kluster baten topologia

- **NODO 1 + BROKER 1** — garapena, probak, ariketa sinpleak
- **NODO 1 + BROKER ANITZ** — kluster baten portaera simulatzeko
- **NODO ANITZ + BROKER ANITZ** — produkzio-inguruneetan ohikoa

Docker Compose - 3 brokerreko klusterra

Broker 1
localhost:9092

Broker 2
localhost:9094

Broker 3
localhost:9095

Broker bakoitzak:

- Broker bakoitzak **ID propioa** dauka
- Broker bakoitzak **bere biltegitratzea** dauka
- Hirurak **Kafka kluster bereko kideak** dira
- Konfigurazioa **compose.yaml** fitxategian definituko da

— TOPIC-A: PARTIZIOAK + ERREPLIKAK

Topic-a sortu: 3 partizio + 2 replication-factor

```
●●● topic-3p2r.sh bash

1 # Topic-a sortu - 3 partizio eta 2 errepikatze-faktore
2 kafka-topics.sh --create --topic iabd-topic-3p2r \
3   --bootstrap-server localhost:9092 \
4   --partitions 3 \
5   --replication-factor 2
6
7 # Topic-a deskribatu
8 kafka-topics.sh --describe --topic iabd-topic-3p2r \
9   --bootstrap-server iabd-virtualbox:9092
10
11 # OHARRAK:
12 #   --partitions 3      -> 3 commit-log paralelo
13 #   --replication-factor 2 -> partizio bakoitzaren 2 kopia
14 #   --bootstrap-server -> edozein broker (metadata-fetch)
15 #   3 broker-eko klusterrean RF=2 -> 3 broker behar lukete
```

Topic-a deskribatu

describe-3p2r.txt

text

```
1 Topic: iabd-topic-3p2r PartitionCount: 3 ReplicationFactor: 2
2
3 Topic: iabd-topic-3p2r Partition: 0 Leader: 1 Replicas: 1,2 Isr: 1,2
4 Topic: iabd-topic-3p2r Partition: 1 Leader: 2 Replicas: 2,3 Isr: 2,3
5 Topic: iabd-topic-3p2r Partition: 2 Leader: 3 Replicas: 3,1 Isr: 3,1
6
7 # INTERPRETAZIOA:
8 #   3 partizio x 2 kopia = 6 replika guztira
9 #   Broker bakoitzak 2 replika ditu (6/3=2)
10 #
11 #   Partizio 0: Leader 1 broker-an, kopia 2-an ere
12 #   Partizio 1: Leader 2 broker-an, kopia 3-an ere
13 #   Partizio 2: Leader 3 broker-an, kopia 1-ean ere
14 #
15 #   LEADER-AK BANATUAK broker-en artean (lan-zama orekatua)
16 #   ISR = Replicas (denak sinkronizatuta – kluster osasuntsua)
```

Producer-a klusterrera konektatu

●●● producer-cluster.py

python

```
1 from kafka import KafkaProducer
2 from json import dumps
3 import time
4
5 # Producer-a HIRU broker-en bootstrap-listarekin
6 producer = KafkaProducer(
7     value_serializer=lambda m: dumps(m).encode('utf-8'),
8     bootstrap_servers=[
9         'localhost:9092',    # broker 1
10        'localhost:9094',    # broker 2
11        'localhost:9095'])   # broker 3
12
13 # 10 mezu round-robin partition-banaketa (gakorik gabe)
14 for i in range(10):
15     producer.send("iabd-topic-3p2r",
16                 value={"izena": "ekoizlea " + str(i)})
17
18 time.sleep(1)    # buffer-bidaltzeko denbora
19 producer.flush() # ziurtatu mezu guztiak bidalita
```

Consumer-a klusterrera konektatu

●●● consumer-cluster.py

python

```
1 from kafka import KafkaConsumer
2 from json import loads
3
4 consumer = KafkaConsumer(
5     'iabd-topic-3p2r',
6     auto_offset_reset='earliest',
7     enable_auto_commit=True,
8     group_id='iabd-taldea-1',
9     value_deserializer=lambda m: loads(m.decode('utf-8')),
10    bootstrap_servers=[
11        'localhost:9092',
12        'localhost:9094',
13        'localhost:9095'])
14
15 # Partizio, offset eta gakoak erakutsi
16 for m in consumer:
17     print(f"P:{m.partition} O:{m.offset} K:{m.key} V:{m.value}")
```

Producer-ak key-rik erabiltzen ez badu:

●●● output-no-key.txt

text

```
1 P:1 O:0 K:None V:{'izena': 'ekoizlea 0'}
2 P:1 O:1 K:None V:{'izena': 'ekoizlea 3'}
3 P:2 O:0 K:None V:{'izena': 'ekoizlea 1'}
4 P:2 O:1 K:None V:{'izena': 'ekoizlea 5'}
5 P:0 O:0 K:None V:{'izena': 'ekoizlea 2'}
6 P:0 O:1 K:None V:{'izena': 'ekoizlea 4'}
7
8 # INTERPRETAZIOA:
9 #   K: None -> mezuak ez du key-rik
10 #   Mezuak partizio desberdinetan egon daitezke
11 #   Offset-a PARTIZIO BAKOITZAREN barruan independentea da:
12 #   Partition 0 -> O:0, O:1...
13 #   Partition 1 -> O:0, O:1...
14 #   Partition 2 -> O:0, O:1...
15 #   Ez dago ordena globalik
16 #   PARTIZIO BAKOITZAREN BARRUAN ordena mantentzen da
```

Key bera → mezu guztiak partizio berera

... producer-key.py

python

```
1 # Producer-aren SEND gakoarekin
2 producer.send("iabd-topic-3p2r",
3     value={"izena": "ekoizlea " + str(i)},
4     key=b"iabd") # gako bera mezu guztietan
5
6 # IRTEERA ORDENATUA (consumer-en):
7 # P:0 O:3 K:b'iabd' V:{'izena': 'ekoizlea 0'}
8 # P:0 O:4 K:b'iabd' V:{'izena': 'ekoizlea 1'}
9 # P:0 O:5 K:b'iabd' V:{'izena': 'ekoizlea 2'}
10 # P:0 O:6 K:b'iabd' V:{'izena': 'ekoizlea 3'}
11 # ...
12 # P:0 O:12 K:b'iabd' V:{'izena': 'ekoizlea 9'}
13
14 # INTERPRETAZIOA:
15 #   Key bera -> partizio bera
16 #   Kasu honetan -> Partition 0
17 #   Offset-ak gorantz doaz partizio horren barruan
18 #   Mezuen ordena mantentzen da key horretarako
```

Kafka Klusterra – ideia nagusiak

- Kluster batek **broker anitz** izan ditzake
- Topic-ak **partiziotan** banatzen dira
- Partizio bakoitzak **Leader** bat dauka
- **Replication Factor**-ak partizio bakoitzaren kopia kopurua adierazten du
- **ISR** -> sinkronizatuta dauden erreplikak
- Producer/Consumer—ek **bootstrap_servers** bidez klusterra aurkitzen dute
- **Key** bera -> **partizio bera** -> **ordena mantendu**

2. KASUA: Kafka Klusterra + Python + NiFi

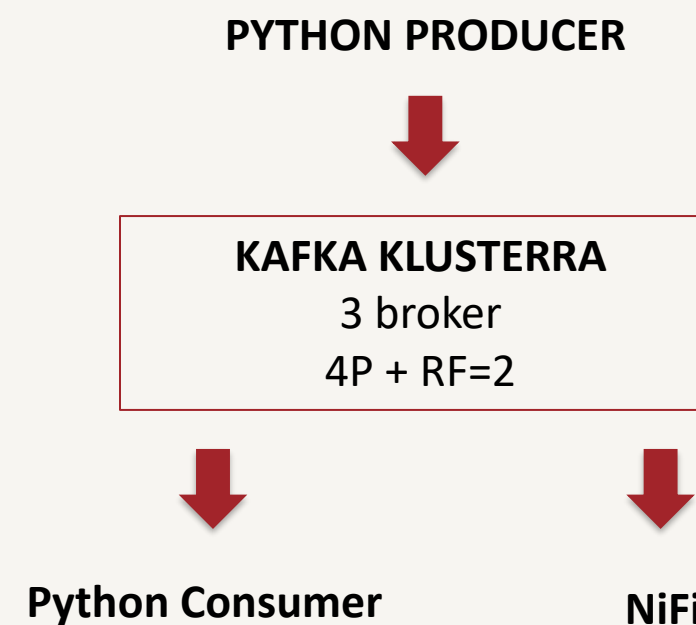
EGOERA

Sortu 3 brokerreko Kafka kluster bat Docker Compose erabiliz.

Sortu topic bat:

- 4 partizio
- Replication Factor = 2

Ondoren, Kafka topic bera hiru tresnarekin erabiliko dugu:



2. KASUA: Kafka Klusterra + Python + NiFi

EGINKIZUNAK

1. Kafka klusterra

- Sortu **compose.yaml** 3 brokerrekin eta abiarazi klusterra

2. Topic-a

- Sortu topic bat: **4 partizio + RF=2**
- Egiaztatu **--describe** erabiliz: **Leader, Replicas** eta **ISR**

3. Python Producer

- Faker erabiliz, sortu **10 pertsona**
- Bidali persona bakoitza Kafka mezu batean
- Datuak: *izena, adina, hiria, emaila*

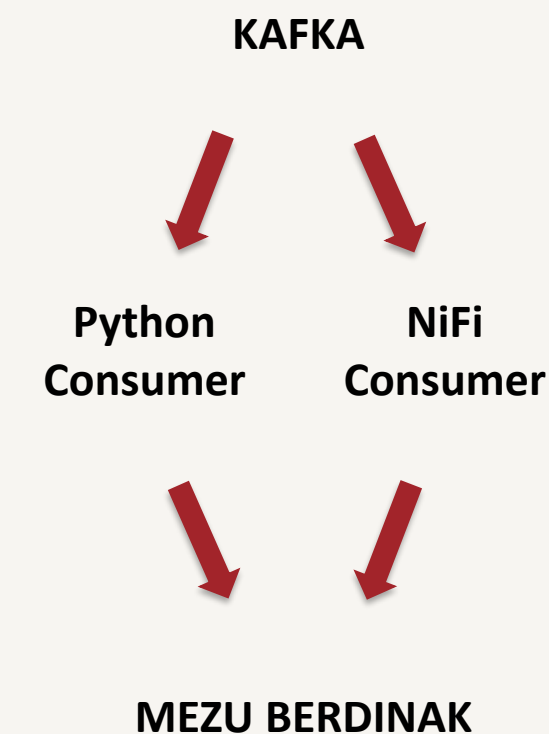
4. Python Consumer

- Kontsumitu topic-a eta erakutsi: **Partition | Offset | Key | Value**

5. NiFi Consumer

- Kontsumitu **topic bera** NiFi-tik
- Erabili Python Consumer-aren **consumer group** desberdina

6. Egiaztatu:



02 Kafka Connect

Kafka Connect

Kafka Connect

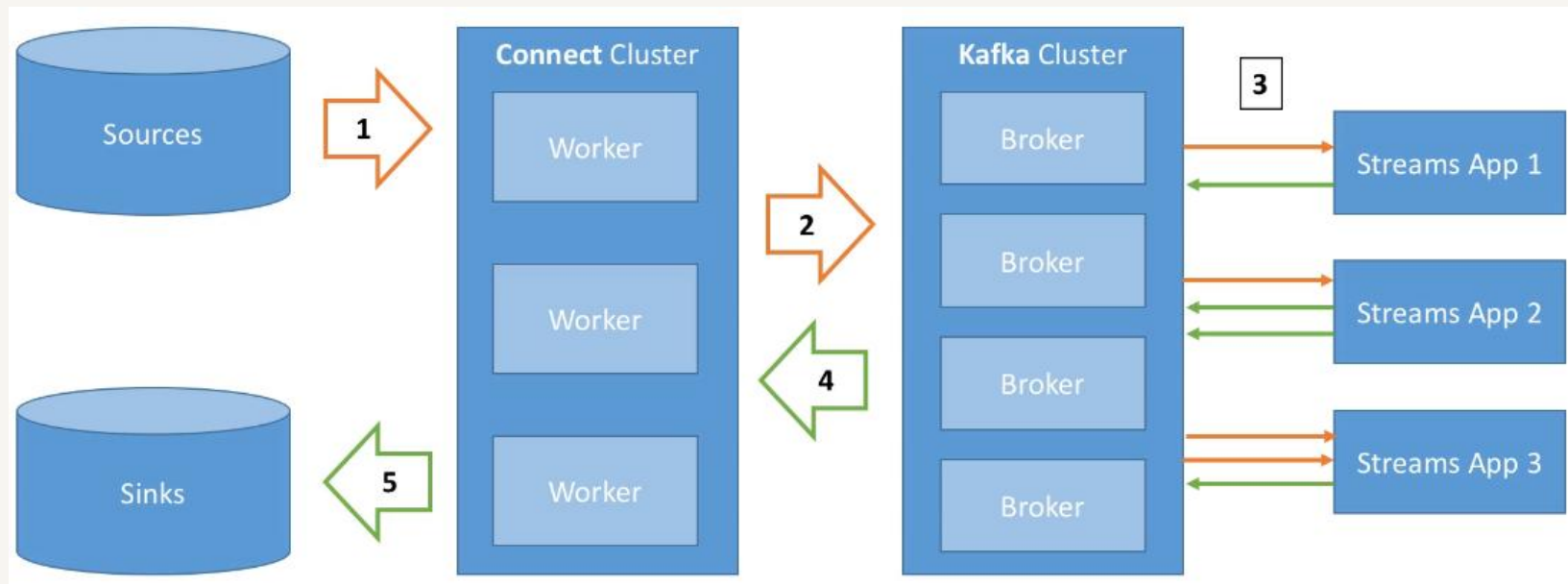
Kafka-n datuak INPORTATZEKO eta ESPORTATZEKO tresna, ITURRI-KONEKTORE (source) eta HELMUGA-KONEKTORE (sink) bidez. KODE IDATZI GABE datuak sistema ezberdinetatik Kafka-ra eta Kafka-tik kanpora transferitzen ditu.

KODE GABE

Konfigurazio-fitxategi-bidez (.properties) soilik — programatzeko-beharrik-gabe. 100+ konektore ofizial: MySQL, MongoDB, S3, Elasticsearch, Salesforce, etab.

— ARKITEKTURA

Kafka Connect — source eta sink konektoreak



- EZKER ALDEAN: source-systems (MySQL, MongoDB, FTP, S3, REST APIs...)
- **SOURCE CONNECTORS** — source-systems-tik Kafka-topic-etara
- **KAFKA** — erdiko mezularitza-hub (topic-etan biltzen)
- **SINK CONNECTORS** — Kafka-topic-etatik downstream-systems-etara
- ESKUIN ALDEAN: sink-systems (Elasticsearch, HDFS, S3, MongoDB, BigQuery...)

MySQL categories tauladunaren egitura

categories-schema.sql

sql

```
1 describe categories;
2
3 -- Field                | Type          | Null | Key
4 -- category_id          | int(11)       | NO   | PRI auto_increment
5 -- category_department_id | int(11)       | NO   |
6 -- category_name        | varchar(45)   | NO   |
7
8 -- HELBURUA:
9 --   Tauladun honen errenkada-guztiak Kafka-topic-batera
10 --   ESPLIZITUKI-PROGRAMATU-GABE — Kafka Connect-ek
11 --   automatikoki egiten du.
12 --
13 -- RETAIL_DB datu-baseko tauladun-bat
14 --   (DataFlow 2-ko 6. kasuan ere erabili genuen)
15 --
16 -- MODE=incrementing
17 --   incrementing.column.name=category_id
18 --   -> Connect-ek category_id-aren-arabera detektatzen
19 --   incrementing.column.name=category_id
```

connect-standalone.properties — worker-config

```
●●● connect-standalone.properties properties

1 # Kafka-broker-en bootstrap (Connect-ek mezuak hor bidaltzen)
2 bootstrap.servers=localhost:9092
3
4 # Key eta Value converter-ak (JSON estandarra)
5 key.converter=org.apache.kafka.connect.json.JsonConverter
6 value.converter=org.apache.kafka.connect.json.JsonConverter
7
8 # Eskema-informazioa mezuetan barneratu (schema + payload)
9 key.converter.schemas.enable=true
10 value.converter.schemas.enable=true
11
12 # Offset-jarraipena fitxategi-batean (standalone mode)
13 offset.storage.file.filename=/tmp/connect.offsets
14
15 # JAR-konektoreen kokapena (deskargatu eta hor ipini)
```

mysql-source-connector.properties

```
●●● mysql-source.properties properties

1 # Konektore-izena (kluster-mailan UNIKOA)
2 name=mysql-source
3
4 # Konektore-klasea — Confluent JDBC source
5 connector.class=io.confluent.connect.jdbc.JdbcSourceConnector
6
7 # Task-paralelismoa (taula-bakoitzeko-task; gehienez taula-kop)
8 tasks.max=1
9
10 # MySQL-konexioa (retail_db datu-basea)
11 connection.url=jdbc:mysql://localhost/retail_db
12 connection.user=iabd
13 connection.password=iabd
14
15 # Zein taula irakurri (whitelist edo blacklist)
16 table.whitelist=categories
17
18 # Detekzio-modua: incrementing (auto-increment-id-arabera)
19 # incremental.mode=incrementing
```

Aurrebaldintzak — plugin eta driver-deskarga

- CONFLUENT JDBC KONEKTOREA deskargatu → /opt/kafka_*/plugins/
- MYSQL DRIVER (mysql-connector-j-X.jar) → /opt/kafka_*/lib/ edo plugin-direktorioan
- **GUAVA LIBURUTEGIA (necesario zenbait bertsiotan)** — ClassPath-ean
- PLUGIN.PATH konfigurazioa → JAR-aurkitze-direktorioa (recursive)
- **CONFLUENT-HUB CLI alternatiba** — 'confluent-hub install <plugin>' komandoarekin automatikoa

Connect abiarazi + topic-a kontsumitu

```
connect-run.sh bash

1 # Connect-standalone abiarazi (worker + konektorea aldi berean)
2 connect-standalone.sh \
3   config/connect-standalone.properties \
4   config/mysql-source-connector.properties
5
6 # TERMINAL ALDI BEREAN - Topic-emaitza kontsumitu
7 kafka-console-consumer.sh \
8   --topic iabd-retail_db-categories \
9   --from-beginning \
10  --bootstrap-server iabd-virtualbox:9092
11
12 # OHARRAK:
13 #   connect-standalone.sh -> worker-process abiarazten
14 #   2. argumentua -> konektore-konfigurazioa
15 #   Topic-izena -> 'iabd-retail_db-' + 'categories'
```

Topic-ean agertzen den JSON-egitura

```
●●● output.json json

1 {
2   "schema": {
3     "type": "struct",
4     "fields": [
5       {"type": "int32", "optional": false, "field": "category_id"},
6       {"type": "int32", "optional": false, "field": "category_department_id"},
7       {"type": "string", "optional": false, "field": "category_name"}
8     ],
9     "optional": false,
10    "name": "categories"
11  },
12  "payload": {
13    "category_id": 1,
14    "category_department_id": 2,
15    "category_name": "Football"
16  }
17 }
```

Kafka Connect REST APIa — 8083 port-ean

```
connect-rest.sh bash

1 # Connect-aren version eta kluster-info
2 curl http://iabd-virtualbox:8083/
3
4 # Eraitza:
5 # {"version":"3.3.1",
6 #   "commit":"839b886f9b732b15",
7 #   "kafka_cluster_id":"iHa0JUnTSfm85fvFadsylA"}
8
9 # Konektore aktiboak zerrendatu
10 curl http://iabd-virtualbox:8083/connectors
11
12 # Eraitza:
13 # ["mysql-source-connector"]
14
15 # Konektore-zehatzaren status-a
16 curl http://iabd-virtualbox:8083/connectors/mysql-source/status
17
18 # Konektore-deploy berria (POST + JSON-body)
```

— SOURCE + SINK BATERA

MySQL → Kafka → MongoDB bide osoa

```
connect-both.sh

1 # Source + Sink konektoreak ALDI BEREAN abiarazi
2 # Standalone-mode-an, argumentu-anitz worker-bakar-batean
3
4 connect-standalone.sh \
5   config/connect-standalone.properties \
6   config/mysql-source-connector.properties \
7   config/mongodb-sink-connector.properties
8
9 # EMAITZA:
10 #   1. mysql-source -> MySQL-categories taula-tik mezuak
11 #       'iabd-retail_db-categories' topic-era
12 #   2. mongodb-sink -> 'iabd-retail_db-categories' topic-etik
13 #       mezuak MongoDB-bilduma-batean idatzi
14 #
15 #   MYSQL-EAN insert-egitean -> AUTOMATIKO MONGODB-EAN agertu
16 #   (Connect-ek bide osoa kudeatzen - kode idatzi gabe!)
17 #
18 #   DISTRIBUTED-MODE-AN hobeagoa fault-tolerance-rako
19 #   (DEFACTO bide bikoizteko erabilgarria da)
```

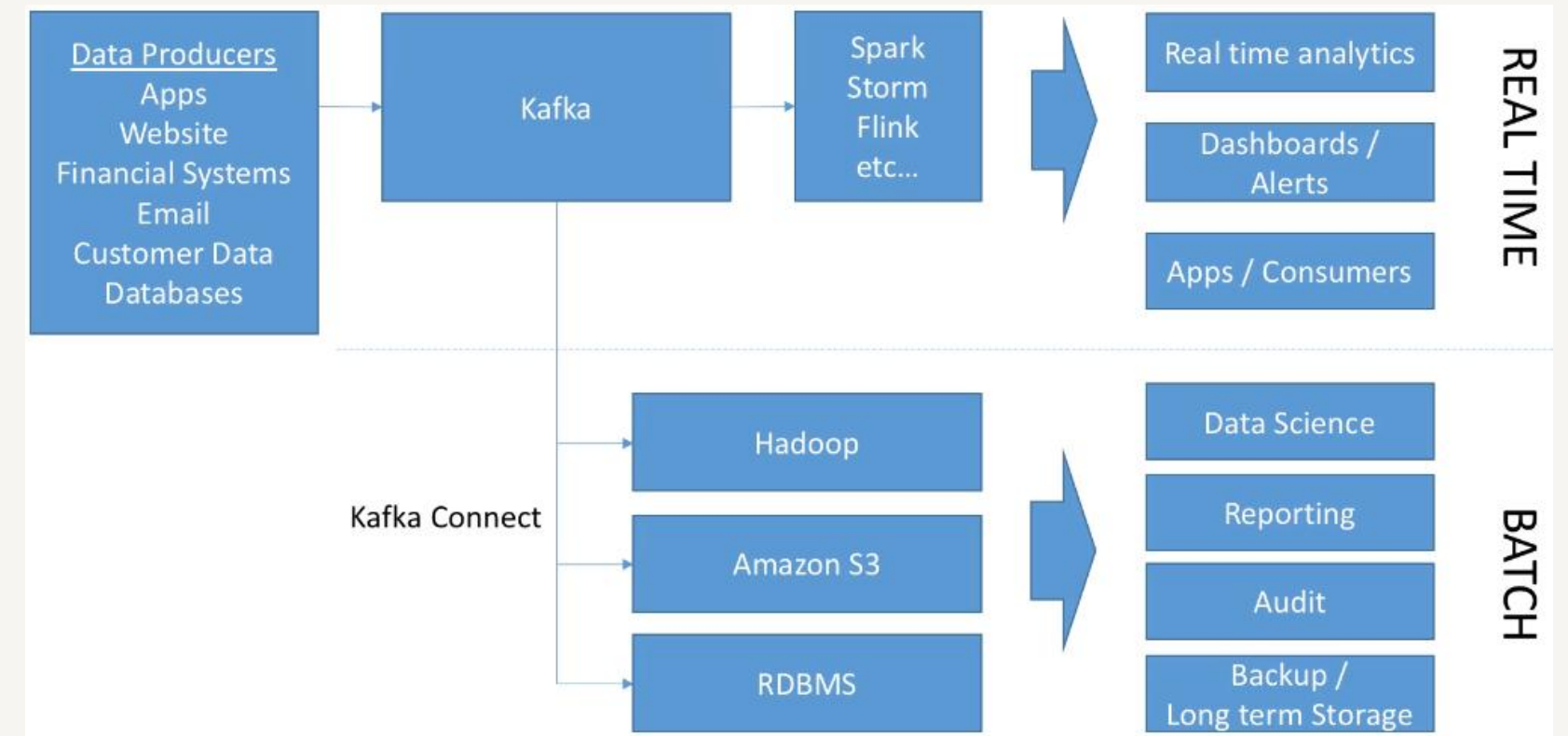
03

Kafka eta Big Data

— KAPPA ARKITEKTURA

Kafka Big Data arkitekturan — batch + streaming

- **KAFKA = MEZULARITZA-HUB ZENTRALA** — datu-iturri-anitzentzat
- **BATCH-INGEST eta STREAMING-INGEST BATERATUTA** — Kafka-bidez bietarako bide-bera
- **KAPPA ARKITEKTURA** — Lambda-aren-alternativa: STREAMING-FIRST, batch-bidearen-beharrik-gabe
- **BIG-DATA-EKOSISTEMA:** Kafka + Spark/Flink + data-lake (S3/HDFS) + DW (Snowflake/BigQuery)
- Kafka Connect = inbiltze-PUNTU NAGUSIA datu-iturri-anitzentzat



Gogoan hartu beharrekoak

-
- i* **Kafka klusterra** — 3 broker minimum produkzioan, broker-ID + port + log-dirs UNIKO-anitz konfigurazio-fitxategi-bana.

 - ii* **Partition + RF** — KLUSTERRA SORTU AURRETIK pentsatu (aldaketak ordena/errendimendua kaltetzen).

 - iii* **Medallion Kafka-en** — Bronze/Silver/Gold TOPIC-INDEPENDENTEAK; pipeline-modularizazioa + consumer-anitzen-aukera.

 - iv* **NiFi + Kafka** — PublishKafka + ConsumeKafka prozesadoreak; record-aldaerak schema-aware produkzio-best-practice.

 - v* **Kafka Connect** — kode-idatzi-gabe datu-iturri-anitz integrazioa; produkzio-ohikoa CDC-rako Debezium-ekin.

 - vi* **Kappa arkitektura** — STREAMING-FIRST eredua; Kafka + Spark/Flink + data-lake produkzio-estandar modernoa.

Galderak, zalantzak, iruzkinak.

Hurrengo saioa: DataFlow 5 — Spark Streaming.

APACHE KAFKA	kafka.apache.org/documentation/
CONNECT	#connect
DOKUMENTAZIOA	
CONFLUENT HUB —	hub.confluent.io
KONEKTORE-	
KATALOGO	
DEBEZIOUM CDC	debezium.io
PLATFORM	
KAPPA	kappa-architecture.com
ARCHITECTURE	
(JAY KREPS)	
JATORRIZKO	Aitor Medrano · CC BY-NC-SA
MATERIALA	